

Trabalho Prático 1

Disciplina Estrutura de Dados – Escalonador de URLs

Raphaela Maria Costa e Silva - 2020006973

Departamento de Ciência da Computação – Universidade Federal de Minas Gerais

Belo Horizonte – MG – Brasil

rapha470@gmail.com

1. Introdução

A proposta do TP foi implementar um programa que escalona URLs usando a estratégia depth-first. Foi sugerida uma implementação de uma fila de listas, na qual os hosts são guardados na fila e os urls nas listas.

Essa documentação tem como objetivo explicar o problema de maneira sucinta, explicar as estruturas de dados e os métodos usados para solucionar o problema, analisar a complexidade de tempo e de espaço do código e apresentar os resultados de análises experimentais em relação ao desempenho computacional e à localidade de referência.

Como solicitado no enunciado do TP, foi implementado uma fila de listas. Os comandos para o programa são passados por um arquivo de texto, que é passado como parâmetro na linha de comando. O programa também gera os dados para a análise experimental a partir da biblioteca ‘memlog’, e os erros do programa são tratados com as macros definidas em ‘msgassert.h’.

2. Método

A implementação do programa teve como base a estrutura de dados fila de listas, uma fila de hosts com listas contendo as urls. Foi escolhida essa estrutura de dados pois nos comandos de adicionar e escalonar seria necessário adicionar hosts em apenas uma extremidade da fila e retirar da outra extremidade (obedecendo o FIFO). Já nas listas, foi levado em consideração que seria necessário inserir em posições diferentes, mas que sempre as urls seriam retiradas de uma mesma extremidade.

A Fila foi implementada com célula início (Chave), célula fim (Chave), e tamanho (int). A implementação da célula início ou célula cabeça reduz o custo assintótico de algumas funções e facilita a iteração dentro da fila. A célula fim é útil para definir casos de parada em loops. Já o tamanho evita a necessidade de iterar a lista inteira para achar a quantidade de membros da fila.

Já a célula (no código é a estrutura Chave), possui host (char*), hostID (int), uma estrutura de ponteiro para o próximo elemento da Fila e sua respectiva Lista. O host guarda o nome do host, o hostID é usado para análise experimental pela biblioteca ‘memlog’. Já a estrutura que aponta para o próximo elemento da Fila é necessária para conectar as Chaves na Fila e iterar pela Fila. Isso torna a estrutura dinâmica, e foi feito dessa forma pois não era possível prever a quantidade de hosts teriam na fila.

As funções das duas estruturas acima são:

Inicialização - inicializa a Fila com valores NULL.

Inserir - cria um elemento Chave e insere no final da fila, também cria a lista para o host e insere o primeiro elemento da lista.

Remover - remove o primeiro elemento da fila.

Exibir - itera toda a fila e escreve no arquivo de saída todos os hosts.

Nexiste - verifica se já tem um certo host na fila, se já tiver insere a url na lista de seu host.

A Lista foi implementada com célula início (No), célula fim (No), e tam (int). A implementação da célula início ou célula cabeça reduz o custo assintótico de algumas funções e facilita a iteração dentro da fila. A célula fim ajuda na inserção de novos elementos, visto que com uma célula fim não é necessário iterar toda a fila para encontrar o último elemento e também possibilitando a implementação de uma lista circular. Já o tamanho evita a necessidade de iterar a lista inteira para achar a quantidade de membros da fila.

Já a célula (no código é a estrutura No), possui url (char*), urlID (int), uma estrutura de ponteiro para o próximo elemento da Lista. O url guarda as urls do host, o urlID é usado para análise de experimental pela biblioteca 'memlog'. Já a estrutura que aponta para o próximo elemento da Lista é necessária para conectar os No's na Lista e iterar pela Lista. Isso torna a estrutura dinâmica, e foi feito dessa forma pois não era possível prever a quantidade de urls teriam em cada lista.

As funções das duas estruturas acima são:

Inserir no início - insere No na posição de início da lista.

Inserir ordenado - insere ou no fim ou no início dependendo da profundidade do url. Ou itera a lista até achar a sua devida posição no meio da lista. Visto que a lista tem No início e No fim, não há necessidade de iterar para inserir nessas posições de início e fim.

Inserir no fim - insere No na posição final da lista.

Remover - remove um No do início da lista.

Imprimir - imprime uma certa quantidade de urls. De modo a iterar pela lista e escrever a url no arquivo de saída.

Não duplicado – itera a lista e se o url já existir retorna falso, se não retorna true.

3. Análise de Complexidade

ADD_URLS

Na função main, o bloco ADD_URLS chama as funções nexiste e inserir. O bloco também realiza diversas outras manipulações de string que não serão consideradas no momento, pois o objetivo é a análise da funcionalidade da estrutura de dados.

A função `nexiste` itera por toda fila até encontrar um certo url. Se ele não encontrar a url a função tem custo $O(n)$, esse seria o melhor caso. No pior caso, a url a ser encontrada estaria na última posição e então seria chamada a função `inserir ordenado`.

A função `inserir ordenado` chama a função `naoDuplicado`, que itera na lista para verificar se algum url já existe na lista, e no seu pior caso ele é $O(n)$. `Inserir ordenado` seu melhor caso chama a função `inserir no início` ou `inserir no fim`, que são funções que não iteram e portanto tem custo $O(1)$. Já em seu pior caso o url teria que ser inserido na penúltima posição, e então seriam iterados $n - 1$ vezes, portanto a função `inserir ordenado` é $O(n)$.

Portanto a função `nexiste` é $O(n) + O(n)$, ou seja, $O(n)$.

Já no caso da função `inserir`, a função tem somente um caso, pois não teria um host para esse url ainda. Portanto, a função cria uma Chave na fila e inicializa seus elementos, essa parte tem custo $O(1)$. Na outra parte, é chamado a função `criar lista e inserir ordenado`. A função `criar lista` somente inicializa a lista com valores NULL, portanto é $O(1)$, nesse caso de `inserir ordenado`, não teria elementos na lista ainda, portanto `inserir ordenado` chamaria `naoDuplicado` (que nesse caso teria custo $O(1)$) e então chamaria `inserir no início`, que como visto antes é $O(1)$.

Como visto acima, o custo assintótico do bloco é $O(n)$.

LISTA_HOSTS

O bloco `LISTA_HOSTS` somente chama a função `exibir`. A função `exibir` itera por toda estrutura da fila e escreve no arquivo de saída, portanto ela é $O(n)$ assim como o bloco.

LIMPA_TUDO

O bloco `LIMPA_TUDO` itera por todos elementos das listas de todas as filas e executa `remover(lista)`. E então ele itera por todos elementos da fila e executa `remover(fila)`.

A função `remover(lista)` remove um elemento sem iterar pela lista, portanto é $O(1)$, e `remover(fila)` também.

Desse modo, o bloco tem custo $O(n*m) + O(m)$, que no caso de o número de hosts ser igual ao número de urls (pior caso), ou seja, no pior caso o bloco é $O(n^2)$.

LIMPA_HOST

O bloco `LIMPA_HOST` itera pela fila até achar determinado host e então chama `remover(lista)` até a lista ficar vazia. No melhor caso (contando que não há erros na entrada e portanto o host procurado existe na fila) o host procurado estaria no começo da fila e então a função `remover(lista)`, que tem custo $O(1)$ como já visto anteriormente, seria chamada n vezes. O pior caso seria se o host estiver na última posição, mas o custo assintótico do pior e do melhor caso seriam iguais, portanto, o bloco tem custo $O(n)$.

VER_HOST

O bloco `VER_HOST` itera pela fila até achar o host procurado e então chama `imprimir` para o caso em que são impressas todas as urls da lista.

A função imprimir itera por toda lista e escreve no arquivo de saída as urls, então a função é $O(n)$.

Portanto, o bloco tem custo assintótico do pior caso igual ao do melhor caso: $O(n) + O(n)$, ou seja, $O(n)$.

ESCALONA_HOST

O bloco ESCALONA_HOST itera pela fila até achar o host procurado, imprime uma quantidade de urls e então chama a função remover(lista) até remover todos urls impressos.

Como visto antes, a função imprimir é $O(n)$ e a função remover(lista) é $O(1)$. No melhor caso, o seria retirado uma url do primeiro host da fila e o custo seria $O(1)$, já no pior caso seriam retirados todas as urls do último host, e o custo seria $O(n) + O(n)$, ou seja, $O(n)$.

ESCALONA_TUDO

O bloco ESCALONA_TUDO itera por toda fila e imprime todos os hosts de todas as filas e então remove todos os elementos de todas as listas.

Ou seja, o bloco seria $O(n^2)$ em seu pior caso, em que o número de hosts e o número de filas é igual.

ESCALONA

O bloco ESCALONA itera por uma determinada quantidade de filas e imprime uma determinada quantidade de urls e então remove essas urls.

O custo assintótico desse bloco no seu pior caso (imprimir todas as urls de todos os hosts e a quantidade de hosts ser igual ao de urls) é $O(n^2)$.

Sumarizando:

Inserir no início, inserir no fim, inicializa, criar lista, inserir, remover(fila), remover(lista) – $O(1)$

Inserir ordenado, nexiste, naoDuplicado, exhibir, imprimir – $O(n)$

ESCALONA_HOST, VER_HOST, LIMPA_HOST, LISTA_HOSTS, ADD_URLS – $O(n)$.

ESCALONA, ESCALONA_TUDO, LIMPA_TUDO – $O(n^2)$

Conclusão:

O custo assintótico relacionado a tempo sem contar com as manipulações de string é $O(n^2)$.

Já contando com as manipulações de string, o custo assintótico de tempo teria relação com o tamanho e quantidade das urls passadas pelo arquivo de entrada e seria calculado no bloco ADD_URLS. No caso em que a quantidade de urls passadas for parecida com o tamanho das URLS o custo assintótico vai ser $O(n^2)$, pois o programa itera por todos ou quase todos caracteres da url de todas as urls.

Já o custo assintótico de espaço, vamos considerar que cada URL tem X caracteres e que a quantidade de hosts e urls são iguais. Então a fila teria um tamanho n vezes X vezes n vezes X. Dessa forma, como X é constante, o programa ocupa espaço $O(n^2)$.

1.4 Estratégias de Robustez

A robustez do programa foi implementada a partir do arquivo 'msgassert.h' usando asserções aplicadas ao código considerando uma condição e uma mensagem de erro.

As condições que levam ao erro são:

Se o arquivo com os comandos de escalonamento não for definido, ou seja, o arquivo passado na linha de comando como argv[1] for inválido.

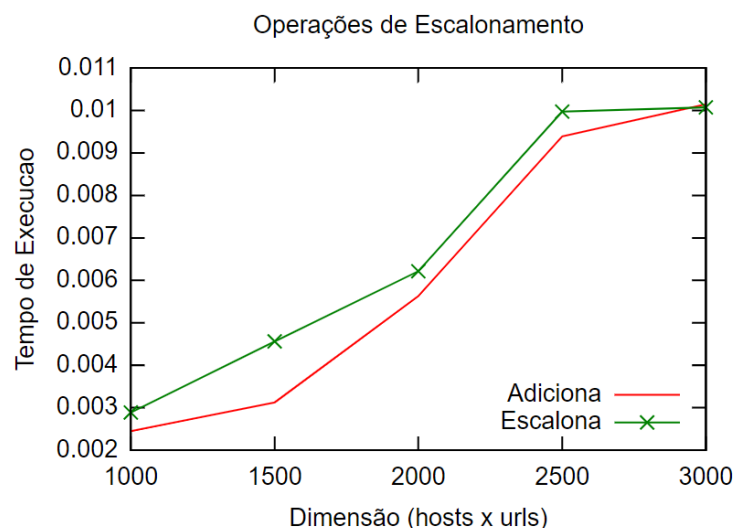
Se o arquivo do registro de acesso de tempo e memória não for definido, ou seja, o arquivo passado na linha de comando como argv[2] for inválido.

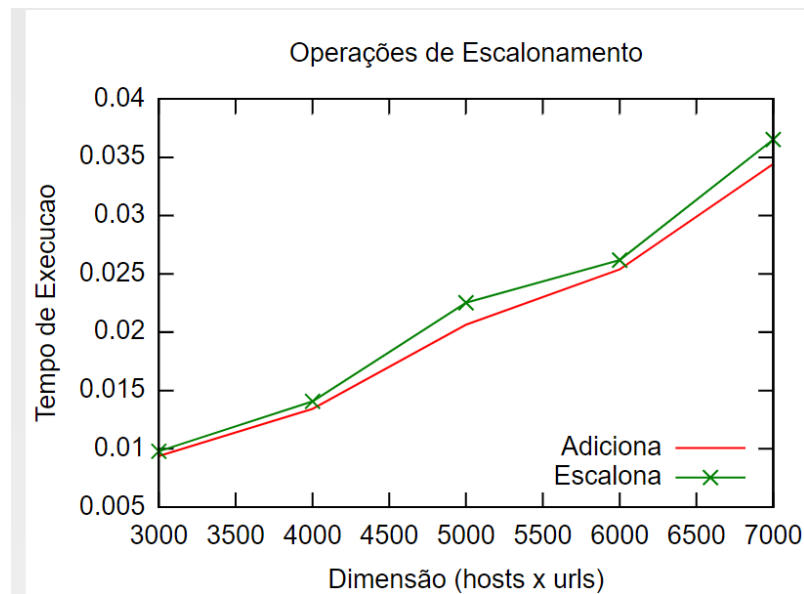
E se haver algum problema com a alocação de memória para o tipo No ou Chave.

Caso algum dos comandos dentro do arquivo passado por argv[1] seja inválido, o programa para e exibe uma mensagem com as opções de uso no terminal.

1.5 Análise Experimental

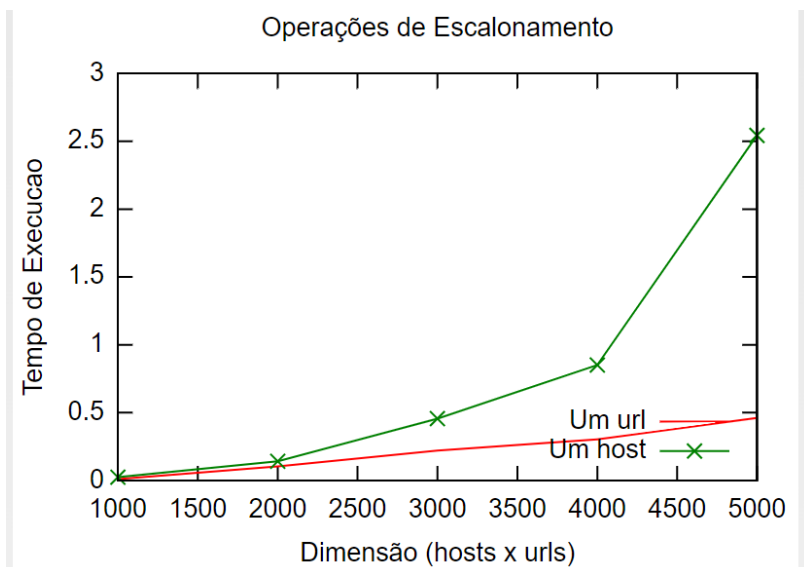
O gráfico abaixo mostra o tempo gasto quando somente adicionamos os hosts e urls no escalonador e depois limpamos tudo, e também mostra quando isso é feito mas com o escalonamento no meio. Nos experimentos foram usadas o mesmo número de elementos em cada filas que a quantidade de hosts.





Visto os gráficos acima, é perceptível que a diferença entre os tempos seria o tempo de escalonamento do código e que o tempo de escalonamento é muito pequeno em relação ao tempo de adicionar e limpar o escalonador e olhando as variações é difícil dizer qual a real contribuição do comando escalonar para o tempo de execução do código.

Mas é possível ver que em conjunto os comandos (adicionar, limpar, e escalonar) tendem a ser $O(n^2)$.



Já esse gráfico nesse gráfico acima, a linha vermelha ocorre quando há apenas um url para cada host, e a linha verde acontece quando há apenas um host e vários urls. Nesse experimento foram usados os comandos adicionar, limpar, listar hosts e escalonar.

Então, é perceptível que quando há apenas um url o tempo de execução aumenta exponencialmente e quando há apenas um host o tempo de crescimento é praticamente linear. Isso provavelmente se deve à iteração feita para percorrer a lista, se há apenas um url em muitas listas, a iteração é pouca, logo o tempo é linear, mas se há muitos elementos na mesma lista, é necessário percorrer todos, e então o tempo aumenta exponencialmente.

1.6 Conclusões

A proposta desse trabalho foi realizar a implementação do escalonador de hosts usando uma fila de listas. Foi um desafio fazer esse trabalho, e implementar essas estruturas de dados. Aprendi sobre como implementar filas e listas e como implementar uma lista circular.

Também aprendi muito sobre alocação de memória e aprendi a usar uma ferramenta chamada Valgrind (ajuda a localizar vazamentos de memória e a localizar erros como funções e variáveis não usadas).

Aprendi a manipular tipos `char*` e `string` de diversas maneiras. De maneira que descobri diversas funções das bibliotecas que manipulam strings.

1.7 Bibliografia

CHAIMOWICS, L and Prates. Slides virtuais e vídeos da disciplina de estrutura de dados. Disponibilizados via moodle nas seções Estrutura de Dados, e Material Adicional.

FARIAS, Ricardo. Estrutura de Dados e Algoritmos. cos.ufrj.br, 2009. Disponível em: <https://www.cos.ufrj.br/~rfarias/cos121/filas.html>.

FEOFILLOFF, Paulo. Listas encadeadas. ime.usp.br, 2018. Disponível em: <https://www.ime.usp.br/~pf/algoritmos/aulas/lista.html>.

FEOFILLOFF, Paulo. Filas. ime.usp.br, 2018. Disponível em: <https://www.ime.usp.br/~pf/algoritmos/aulas/fila.html>.

SANSON, André. Como mostrar/exibir uma lista de strings em lista encadeada. pt.stackandoverflow.com, 2021. Disponível em: <https://pt.stackoverflow.com/questions/410020/como-mostrar-exibir-uma-lista-de-strings-em-lista-encadeada>.

1.8 Instruções para compilação e execução

O programa foi desenvolvido na linguagem C++, compilada por G++ da GNU Compiler Collection no SO Linux.

Acesse o diretório TP. Compile os arquivos usando o comando make. O executável gerado estará em bin. Agora entre no diretório bin e execute o programa.

Para executar o programa deve-se passar na linha de comando o arquivo com os comandos de escalonamento e o arquivo onde será feito o registro de acesso. A terceira opção de argumento na linha de parâmetro é facultativo, escreva 'l' caso desejar que seja registrado os acessos à memória, não escreva nada caso quiser que seja registrado somente o tempo de execução.

Os arquivos passados pela linha de comando devem estar no diretório bin.

O arquivo com a saída aparecerá no diretório bin com a seguinte formatação: "arquivoDeEntrada-out.txt".

Por exemplo: `./main entrada.txt análise.txt l`

O arquivo de saída seria o "entrada-out.txt" e será registrado os acessos à memória.